

Opis Struktury Danych $\sqrt{n}$

Struktura polega na podzieleniu n elementów na $\sqrt{n}$ bloków, gdzie każdy blok jest posortowany rosnąco oraz dla dowolnych bloków i, j , gdzie $i < j$, wszystkie elementy bloku j są większe od elementów bloku i . W każdym bloku znajduje się od $\sqrt{n}$ do $2\sqrt{n}$ elementów.

Operacje

- **push(x)** – Szuka liniowo po $\sqrt{n}$ blokach bloku, którego ostatni element jest większy niż x , a następnie umieszcza element na odpowiedniej pozycji w tym bloku, tak aby był on posortowany również w czasie $O(\sqrt{n})$. Cała procedura zajmuje zatem $O(2\sqrt{n})$.
- **find(x)** – Szukanie bloku odbywa się liniowo po $\sqrt{n}$ blokach, a szukanie wewnątrz bloku również w $O(\sqrt{n})$.
- **del(x)** – Szukanie bloku w $O(\sqrt{n})$, a usunięcie brutalnie również w $O(\sqrt{n})$.
- **print(x)** – Wypisywanie w $O(n)$ polegające na wypisywaniu zawartości kolejnych bloków.

Oprócz tego co `sqrt(n)` (aktualnego w momencie wcześniejszej dekompozycji) operacji następuje dekompozycja wszystkich bloków i w czasie $O(N)$ tworzone jest dokładnie $\lceil \sqrt{n} \rceil$ nowych bloków zawierających dokładnie $\lfloor \sqrt{n} \rfloor$ elementów (poza ostatnim). Taki zabieg sprawia, że w najgorszym wypadku dodanych (lub usuniętych) zostanie $\sqrt{n}$ elementów, rozmieszczonych po $\sqrt{n}$ blokach co nie wpłynie na złożoność powyższych operacji. Natomiast fakt, że wykonujemy ową dekompozycję co $\sqrt{n}$ operacji sprawia, że zamortyzowany czas na jedną operację wynosi $O(n/\sqrt{n}) = O(\sqrt{n})$